

PROJECT REPORT

EXPERIMENT - 3

Project title

**IoT-Based HVAC Temperature and Humidity Monitoring
System**

Submitted by

Abinaya S – 24MER001

Dhiyanasree KK – 24MER008

Puthiyaraj M – 24MER045

Rithick S – 24MER051

IoT-Based HVAC Temperature and Humidity Monitoring System

Project Overview

The IoT-Based HVAC Temperature and Humidity Monitoring System is designed to continuously monitor the environmental temperature and humidity of a room and support efficient HVAC operation. The system uses a DHT22 sensor to measure temperature and humidity, while the ESP8266 Node MCU acts as the main controller.

The ESP8266 receives the sensor readings and displays the temperature and humidity values on an LCD display. When the measured environmental conditions exceed predefined limits, the ESP8266 activates the relay module and buzzer according to the programmed control logic. The ESP8266 also provides Wi-Fi connectivity for future IoT-based remote monitoring.

This system helps reduce unnecessary HVAC operation, improve environmental monitoring, provide alerts, and support better energy management.

Main Objectives:

- To continuously monitor room temperature using the DHT22 sensor.
- To continuously monitor room humidity using the DHT22 sensor.
- To display real-time temperature and humidity values.
- To detect abnormal temperature and humidity conditions.
- To automatically control the relay based on predefined conditions.
- To provide an audible warning using the buzzer.
- To use ESP8266 for IoT connectivity.
- To support efficient HVAC operation and energy management.
- To reduce unnecessary HVAC operation.
- To develop a simple and low-cost HVAC monitoring prototype.

Problem Statement

Traditional HVAC systems may operate without continuously considering the actual temperature and humidity conditions of the environment. This can result in unnecessary HVAC operation, increased energy consumption, poor environmental control, and lack of real-time monitoring.

Users may also need to manually monitor environmental conditions and adjust the HVAC system accordingly.

To overcome these problems, an IoT-Based HVAC Temperature and Humidity Monitoring System is proposed. The system continuously measures temperature and humidity using a DHT22 sensor.

When the measured values exceed predefined limits, the system activates the relay and buzzer according to the programmed control logic. The ESP8266's Wi-Fi capability can also be used for IoT-based remote monitoring.

Problems Addressed:

The proposed IoT-Based HVAC Temperature and Humidity Monitoring System addresses the following problems:

- **Manual Monitoring** – Reduces the need for continuous manual checking of environmental conditions.
- **Excessive HVAC Operation** – Helps control HVAC operation based on actual environmental conditions.
- **Energy Wastage** – Helps reduce unnecessary HVAC operation and supports energy management.
- **Temperature Monitoring** – Continuously measures room temperature using the DHT22.
- **Humidity Monitoring** – Continuously measures room humidity.
- **Abnormal Conditions** – Detects conditions outside predefined limits.
- **Lack of Alerts** – Provides an audible warning using the buzzer.
- **Lack of Automation** – Provides automatic relay control based on programmed conditions.
- **Lack of IoT Connectivity** – Uses ESP8266 Wi-Fi capability for future remote monitoring.
- **Poor Environmental Management** – Provides continuous environmental data for better HVAC management.

Proposed Solution

The proposed solution is to develop an IoT-Based HVAC Temperature and Humidity Monitoring System using ESP8266.

The system will:

- Use the DHT22 sensor to measure temperature and humidity.
- Use the ESP8266 Node MCU as the main controller.
- Process the sensor readings using programmed threshold values.
- Activate the buzzer when an abnormal condition is detected.
- Control the relay module according to the programmed HVAC control condition.
- Use Wi-Fi connectivity for IoT-based monitoring.
- Continuously repeat the monitoring process.
- Support improved HVAC energy management.

Proposed Operation:

- The DHT22 continuously senses room temperature and humidity.
- The ESP8266 Node MCU receives the sensor readings.
- The ESP8266 processes the temperature and humidity values.
- The temperature and humidity values are compared with predefined limits.
- If the values are within the desired range, the system maintains normal operation.
- If the temperature or humidity exceeds the predefined limit, the buzzer is activated.
- The ESP8266 changes the relay state according to the programmed HVAC control logic.
- The environmental values can be transmitted through Wi-Fi for IoT monitoring.
- The process continuously repeats.

System Architecture

Input Section:

- **DHT22 Sensor** – Measures room temperature and humidity.

Processing and Control Section:

- **ESP8266 Node MCU** – Receives sensor data, processes the readings, compares them with thresholds, and controls the relay and buzzer.
- **Relay Module** – Acts as an electronic switching interface for HVAC control.

Output Section:

- **Buzzer** – Provides an audible warning during abnormal conditions.

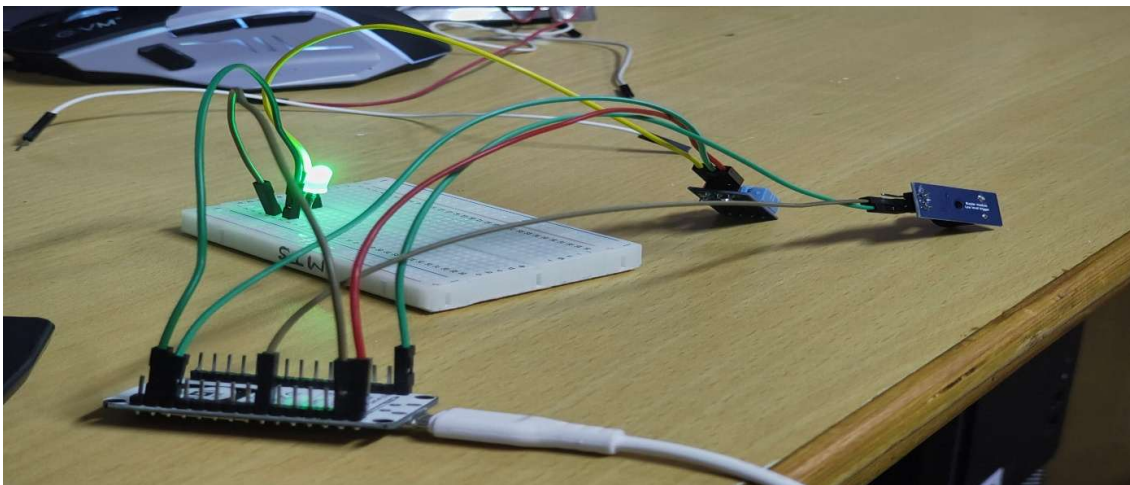
IoT Section:

- **ESP8266 Wi-Fi** – Provides wireless connectivity for future remote monitoring and data logging.

Basic Flow:

- **ESP8266 → Relay → HVAC Control**
- **ESP8266 → Buzzer → Alert**
- **ESP8266 → Wi-Fi → IoT Monitoring**

Image:



Circuit Table

Component	Pin	ESP8266 Node MCU Connection
DHT22 Sensor	VCC	3.3V
	GND	GND
	DATA	D4 (GPIO2)
Relay Module	VCC	Suitable supply
	GND	GND
	IN	D5 (GPIO14)
Buzzer	+	D6 (GPIO12)
	–	GND

Important Pin Mapping:

DHT22 Sensor

- VCC → 3.3V
- GND → GND
- DATA → D4 (GPIO2)

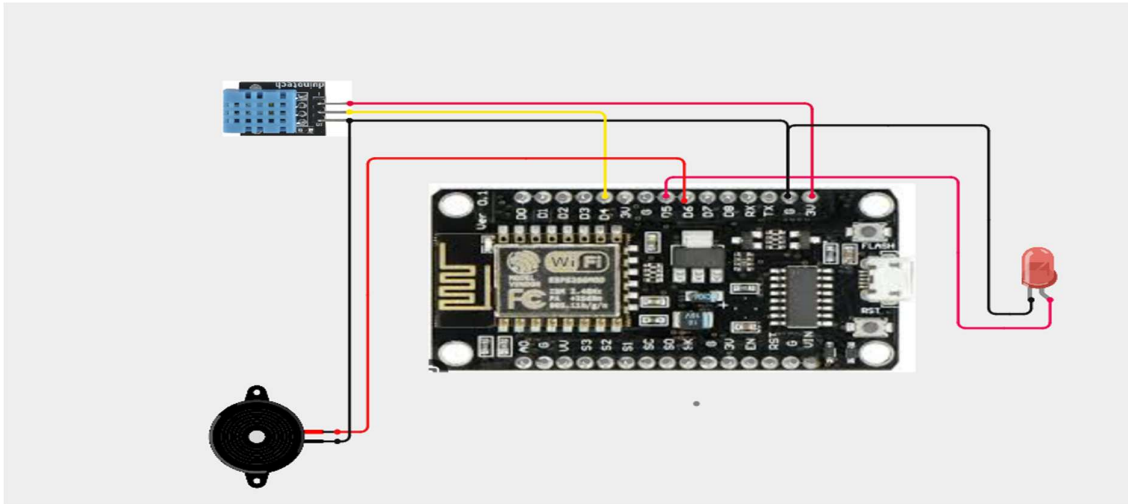
Relay Module

- IN → D5 (GPIO14)
- VCC → Suitable supply
- GND → GND

Buzzer

- Positive → D6 (GPIO12)
- Negative → GND

Circuit Design



Algorithm

Step 1: Start the system.

Step 2: Initialize the ESP8266 Node MCU.

Step 3: Initialize the DHT22 sensor.

Step 4: Initialize the LCD display.

Step 5: Initialize the relay and buzzer.

Step 6: Connect the ESP8266 to Wi-Fi.

Step 7: Read temperature from the DHT22.

Step 8: Read humidity from the DHT22.

Step 9: Display temperature and humidity on the LCD.

Step 10: Compare the readings with predefined threshold values.

Step 11: If the environmental condition is abnormal, activate the buzzer.

Step 12: Activate or deactivate the relay according to the programmed HVAC control logic.

Step 13: Send the readings through Wi-Fi if IoT monitoring is enabled.

Step 14: Repeat the monitoring process continuously.

Coding

```
#include <ESP8266WiFi.h>

#include <ESP8266HTTPClient.h>

#include <DHT.h>


// =====

// WiFi

// =====

const char* ssid    = "Rithick";
const char* password = "12345678";


// =====

// ThingSpeak

// =====

const char* thingspeakServer = "http://api.thingspeak.com/update";
const char* apiKey = "BAW3XHR6BKMJNWBW";


// =====

// PIN DEFINITIONS - NodeMCU ESP8266

// =====

//

// D2 = GPIO4 -> DHT11 DATA
// D1 = GPIO5 -> LED
// D6 = GPIO12 -> Buzzer

//

// =====
```

```
#define DHTPIN    04

#define DHTTYPE   DHT11


#define LED_PIN   05
#define BUZZER_PIN 12
DHT dht(DHTPIN, DHTTYPE);


// =====
// HVAC LIMITS
// =====


const float TEMP_HIGH = 30.0;
const float TEMP_LOW  = 20.0;


const float HUM_HIGH  = 70.0;
const float HUM_LOW   = 40.0;


// =====
// ThingSpeak timing
// =====


unsigned long lastUpload = 0;


// ThingSpeak requires at least 15 seconds between updates.
// We use 20 seconds.
```

```
const unsigned long uploadInterval = 20000;

// =====

// SETUP

// =====

void setup()
{
  Serial.begin(115200);
  delay(100);

  Serial.println();
  Serial.println("=====");
  Serial.println(" ESP8266 HVAC MONITOR");
  Serial.println("=====");

  // -----

  // GPIO setup

  // -----

  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  // LED OFF

  digitalWrite(LED_PIN, LOW);
```

```
// Active LOW buzzer -> OFF
digitalWrite(BUZZER_PIN, HIGH);

// -----
// Start DHT11
// -----

dht.begin();

Serial.println("DHT11 initialized");

// -----
// Connect WiFi
// -----

WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);

Serial.print("Connecting to WiFi");

unsigned long startTime = millis();

while (WiFi.status() != WL_CONNECTED &&
      millis() - startTime < 20000)
{
```

```
    delay(500);
    Serial.print(".");
}

Serial.println();

if (WiFi.status() == WL_CONNECTED)
{
    Serial.println("WiFi Connected!");

    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
}
else
{
    Serial.println("WiFi connection failed.");
    Serial.println("Device will continue and retry later.");
}

Serial.println("=====");
}

// =====
// CONNECT WIFI IF DISCONNECTED
// =====
```

```
void connectWiFi()
{
  if (WiFi.status() == WL_CONNECTED)
  {
    return;
  }

  Serial.println("WiFi disconnected.");
  Serial.println("Attempting reconnection...");

  WiFi.disconnect();
  delay(100);

  WiFi.begin(ssid, password);

  unsigned long startTime = millis();

  while (WiFi.status() != WL_CONNECTED &&
        millis() - startTime < 10000)
  {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
```

```
if (WiFi.status() == WL_CONNECTED)
{
    Serial.println("WiFi reconnected!");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
}
else
{
    Serial.println("WiFi reconnect failed.");
}
}
```

```
// =====
// HVAC ALERT
// =====
```

```
void checkHVAC(float temperature, float humidity)
{
    bool alert = false;

    // Temperature limits
    if (temperature > TEMP_HIGH ||
        temperature < TEMP_LOW)
    {
```

```
    alert = true;
}

// Humidity limits
if (humidity > HUM_HIGH ||
    humidity < HUM_LOW)
{
    alert = true;
}

if (alert)
{
    Serial.println("!!! HVAC ALERT !!!");

    // LED ON
    digitalWrite(LED_PIN, HIGH);

    // Active LOW buzzer ON
    digitalWrite(BUZZER_PIN, LOW);
}
else
{
    Serial.println("Environment Normal");

    // LED OFF
    digitalWrite(LED_PIN, LOW);
}
```

```

    // Active LOW buzzer OFF
    digitalWrite(BUZZER_PIN, HIGH);
}
}

// =====

// UPLOAD TO THINGSPEAK

// =====

void uploadThingSpeak(float temperature, float humidity)
{
    if (WiFi.status() != WL_CONNECTED)
    {
        Serial.println("ThingSpeak upload skipped - WiFi disconnected.");
        return;
    }

    WiFiClient client;
    HTTPClient http;

    String url = String(thingspeakServer) +
        "?api_key=" + apiKey +
        "&field1=" + String(temperature, 1) +
        "&field2=" + String(humidity, 1);

```

```
Serial.println("Uploading to ThingSpeak...");
Serial.println(url);

// Start HTTP connection
if (!http.begin(client, url))
{
    Serial.println("HTTP begin failed.");
    return;
}

// Timeout prevents the ESP8266 from waiting indefinitely
http.setTimeout(5000);

int httpCode = http.GET();

if (httpCode > 0)
{
    Serial.print("ThingSpeak HTTP Response: ");
    Serial.println(httpCode);

    String response = http.getString();

    Serial.print("ThingSpeak Response: ");
    Serial.println(response);
}
```

```
else
{
  Serial.print("ThingSpeak upload failed. Error: ");
  Serial.println(http.errorToString(httpCode));
}
```

```
http.end();
}
```

```
// =====
// MAIN LOOP
// =====
```

```
void loop()
{
  // -----
  // Make sure WiFi is connected
  // -----
```

```
connectWiFi();
```

```
// -----
// Read DHT11
// -----
```

```
float temperature = dht.readTemperature();
float humidity    = dht.readHumidity();

// -----
// Check DHT11 reading
// -----

if (isnan(temperature) || isnan(humidity))
{
    Serial.println();
    Serial.println("ERROR: Failed to read DHT11!");

    // Keep outputs in safe state
    digitalWrite(LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, HIGH);

    delay(2000);
    return;
}

// -----
// Display readings
// -----

Serial.println();
Serial.println("-----");
```

```
Serial.print("Temperature : ");  
Serial.print(temperature, 1);  
Serial.println(" °C");
```

```
Serial.print("Humidity  : ");  
Serial.print(humidity, 1);  
Serial.println(" %");
```

```
// -----  
// HVAC alert  
// -----
```

```
checkHVAC(temperature, humidity);
```

```
// -----  
// ThingSpeak upload  
// -----
```

```
if (millis() - lastUpload >= uploadInterval ||  
    lastUpload == 0)  
{  
    uploadThingSpeak(temperature, humidity);  
  
    lastUpload = millis();  
}
```

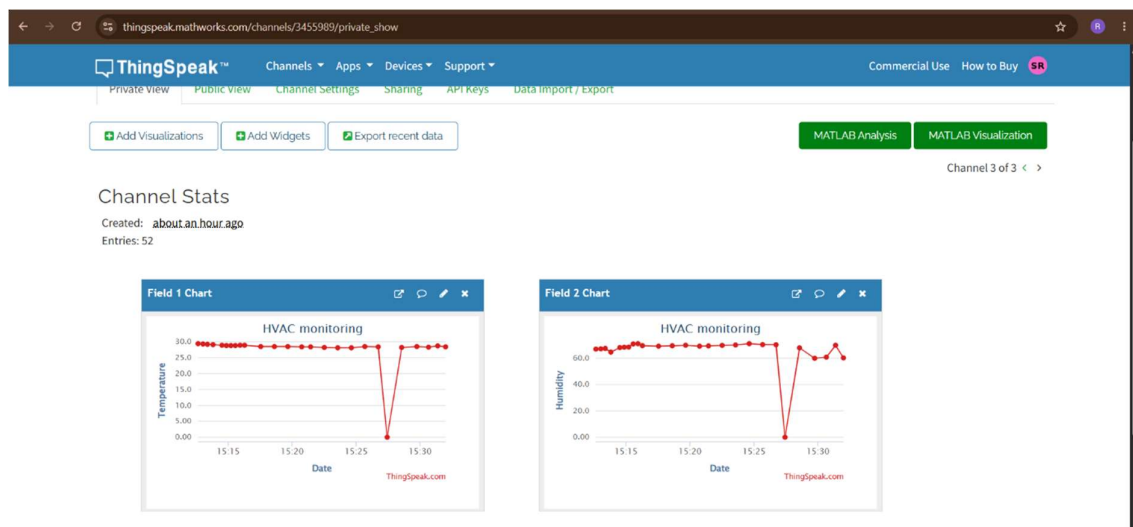
```
Serial.println("-----");
```

```
// DHT11 should not be read too rapidly
```

```
delay(2000);
```

```
}
```

Coding Output:



```
sketch_aug14e | Arduino IDE 2.3.10
File Edit Sketch Tools Help

Generic ESP8266 Mod...

sketch_aug14e.ino
9  const char* password = "12345678";
10
11  // -----
12  // Thingspeak
13  // -----
14  const char* thingspeakServer = "http://api.thingspeak.com/update";
15  const char* apiKey = "BAW43XR68RQD1M6W";
16

Output Serial Monitor X
Message (Enter to send message to 'Generic ESP8266 Module' on 'COM10') New Line 115200 baud

-----
Temperature : 28.9 °C
Humidity : 70.3 %
!!! HVAC ALERT !!!
-----

Temperature : 28.9 °C
Humidity : 70.1 %
!!! HVAC ALERT !!!
-----

Temperature : 28.9 °C
Humidity : 70.0 %
Environment Normal
-----

Temperature : 28.9 °C
Humidity : 69.8 %
Environment Normal
-----

Ln 20, Col 3 Generic ESP8266 Module on COM10 2
```

System Working

- The system is powered ON.
- The ESP8266 initializes the DHT22, LCD, relay and buzzer.
- The ESP8266 establishes a Wi-Fi connection.
- The DHT22 measures the surrounding temperature.
- The DHT22 measures the surrounding humidity.
- The ESP8266 receives both readings.
- The readings are displayed on the LCD.
- The ESP8266 compares the readings with the predefined limits.
- If the temperature or humidity exceeds the limit, the buzzer generates an alert.
- The relay changes its state according to the programmed HVAC control logic.
- The environmental values can be monitored through an IoT platform when connected.
- The system continuously repeats this process.

Bill of Materials

S. No.	Component	Quantity
1	ESP8266 Node MCU	1
2	DHT22 Temperature & Humidity Sensor	1
3	Relay Module	1
4	Buzzer	1
5	Breadboard	1
6	Jumper Wires	1 Set

Prototype Implementation

The IoT-Based HVAC Temperature and Humidity Monitoring System is implemented using an ESP8266 Node MCU, DHT22 sensor, relay module, LCD display, and buzzer.

- **Hardware Assembly:** The DHT22 sensor, display, relay and buzzer are connected to the ESP8266 using the breadboard and jumper wires.
- **Environmental Monitoring:** The DHT22 continuously measures temperature and humidity.
- **Data Processing:** The ESP8266 processes the sensor readings.
- **Display:** Temperature and humidity values are displayed on the LCD.
- **Alert System:** The buzzer is activated when the measured values exceed the predefined limits.
- **HVAC Control:** The ESP8266 controls the relay according to the programmed environmental conditions.
- **IoT Connectivity:** The ESP8266 connects to Wi-Fi for possible remote monitoring.
- **Testing:** The prototype is tested under normal and high-temperature/humidity conditions to verify the monitoring, alert and relay operation.

Hardware Setup:

The prototype hardware is assembled as follows:

- **ESP8266 Node MCU** – Main controller for processing sensor readings and controlling the outputs.
- **DHT22 Sensor** – Measures temperature and humidity.
- **LCD Display** – Displays real-time environmental readings.
- **Relay Module** – Provides the switching interface for HVAC control.
- **Buzzer** – Provides an audible warning during abnormal conditions.
- **Breadboard & Jumper Wires** – Used to construct the prototype circuit.
- **Power Supply** – Provides suitable power to the ESP8266 and connected modules.

Prototype Working:

- The system is powered ON.
- The ESP8266 initializes all connected components.
- The DHT22 detects the surrounding temperature and humidity.
- The sensor sends the readings to the ESP8266.
- The ESP8266 processes the readings.
- The current temperature and humidity are shown on the LCD.
- When the readings are within the predefined range, the system operates in normal mode.
- When temperature or humidity exceeds the predefined limit, the buzzer is activated.
- The relay changes state according to the programmed HVAC control condition.
- The ESP8266 can transmit the readings through Wi-Fi.
- The process repeats continuously for real-time monitoring.

Expected Prototype Output:

Condition	Temperature/ Humidity Reading	Display	Buzzer	Relay	System Action
Normal Environment	Within limits	Normal values	OFF	Normal	Normal monitoring
High Temperature	Above temperature limit	High temperature	ON	Control activated	HVAC control
High Humidity	Above humidity limit	High humidity	ON	Control activated	HVAC control
High Temperature & Humidity	Both above limits	Warning	ON	Control activated	Alert and HVAC control
Normal Condition Restored	Within limits	Normal values	OFF	Normal	Normal operation

Prototype Demonstration:

The prototype is powered ON, and the ESP8266 initializes the DHT22 sensor, LCD display, relay module and buzzer. The DHT22 continuously measures the surrounding temperature and humidity.

The measured values are displayed on the LCD. Under normal environmental conditions, the system displays the measured values and maintains normal operation.

When the temperature or humidity is artificially increased beyond the predefined threshold, the ESP8266 detects the abnormal condition. It activates the buzzer and changes the relay state according to the programmed control logic.

When the environmental condition returns to the acceptable range, the system returns to normal operation. This demonstration verifies the temperature and humidity monitoring, alert generation and automatic HVAC control functionality.

Results & Performance Analysis

The developed IoT-Based HVAC Temperature and Humidity Monitoring System successfully monitors environmental temperature and humidity using the DHT22 sensor. The ESP8266 receives and processes the sensor data and displays the readings on the LCD.

When the measured environmental conditions exceed the predefined limits, the buzzer provides an alert and the relay responds according to the programmed control logic. The ESP8266 also provides Wi-Fi connectivity for IoT-based monitoring.

The prototype demonstrates continuous environmental monitoring, automatic condition-based control, alert generation and the potential for improved HVAC energy management.

Expected Results:

- Automatically measures room temperature.
- Automatically measures room humidity.
- Displays real-time readings on the LCD.
- Detects high-temperature conditions.

- Detects high-humidity conditions.
- Activates the buzzer during abnormal conditions.
- Controls the relay according to predefined conditions.
- Provides Wi-Fi connectivity through ESP8266.
- Supports IoT-based remote monitoring.
- Reduces unnecessary HVAC operation.
- Supports improved energy management.
- Provides continuous environmental monitoring.
- Minimizes the need for manual monitoring.
- Provides reliable prototype operation.

Performance Analysis:

Parameter	Performance	Observation
Temperature Monitoring	Good	DHT22 continuously measures temperature
Humidity Monitoring	Good	DHT22 provides continuous humidity readings
Display Operation	Good	OLED/LCD displays real-time values
Abnormal Condition Detection	Good	System detects readings above predefined limits
Buzzer Alert	Reliable	Buzzer provides warning during abnormal conditions
Relay Control	Reliable	Relay responds according to programmed logic
IoT Connectivity	Good	ESP8266 provides Wi-Fi connectivity
Response Time	Fast	System responds quickly to sensor changes
Continuous Monitoring	Effective	Environmental conditions are monitored continuously
Energy Management Support	Effective	Condition-based control can reduce unnecessary HVAC operation
Overall System Performance	Satisfactory	Provides reliable temperature/humidity monitoring and control